动态规划自学讲义

第二册 提高组建模与优化

从已知模型走向陌生条件

这一册默认你已能独立完成第一册的线性、网格、背包与序列问题。遇到新题时，仍先写“未来需要知道什么”，再决定状态。二维或三维只是在记更多必要信息，树形、区间、集合也只是不同的依赖形状。

第 12 至 20 章训练模型，第 21 至 23 章训练优化，第 24 章把方法放回陌生题。前置知识会在各章标明。不认识树、位运算或概率时先完成前置小课。不要把第一次看不懂高阶内容当成前面的学习没有价值。

每章的公式都是推导结果。先画小例子再读公式，完成独立任务后再去第三册做不标模型的练习。模型学习与练习验收分开进行。

本册目录

第 12 章 区间问题从最后一次合并入手 3

第 13 章 依赖图和有向无环图 4

第 14 章 把有限的历史变成状态机 5

第 15 章 多维状态和两个人同步走 6

第 16 章 树形动态规划先认识父子关系 7

第 17 章 换根把一棵树的答案传给邻居 9

第 18 章 把小集合写进一个整数 10

第 19 章 数位动态规划把大范围拆成短数字 11

第 20 章 概率与期望也是对子问题的合并 12

第 21 章 优化之前先找到重复劳动 13

第 22 章 单调队列维护会过期的候选 14

第 23 章 进一步优化要先证明条件 16

第 24 章 面对没有标签的新题 18

本册资料说明 20

目录可点击跳转。打印阅读时可按章选择页码。

第 12 章 区间问题从最后一次合并入手

相邻石子为什么不是普通背包

有一排石子堆，数量为 4、1、3、2。每次只能把相邻的两堆合成一堆，代价等于这两堆的石子总数。反复合并成一堆，求总代价最小。一次合并后，新堆还会参加以后的合并，石子可能被重复计入代价。

先停下来想：最后一次合并发生前，原来的区间会变成什么样？它一定由左右两段组成吗？把四堆画成三种最后分界，分别在第 1、2、3 堆后分开。

定义 dp[l][r]为把第 l 至 r 堆全部合成一堆的最小代价。最后一次合并前，必定是[l,k]与[k+1,r]各自先合成一堆。它们互不重叠，正好覆盖原区间，最后再付整个区间的石子总数。

$$dp[l][r] = \min(dp[l][k] + dp[k+1][r] + sum(l,r))$$

其中 l<=k<r。单堆不用合并，dp[i][i]=0。区间越长，依赖越短，因此先算长度 2，再算长度 3，最后长度 n。时间 O(n³)：约 n² 个区间，每个枚举约 n 个分界。空间 O(n²)。

区间	最后合并前的候选代价	最小代价
1 至 2	4+1	5
2 至 3	1+3	4
3 至 4	3+2	5
1 至 3	0+4+8 或 5+0+8	12
2 至 4	0+5+6 或 4+0+6	10
1 至 4	0+10+10 或 5+5+10 或 12+0+10	20

第 1 至 4 区间总数是 10，但最终最小代价是 20，因为有些石子在早先合并中也付过代价。“结果的一堆有多大”与“过程中累计付了多少”是不同变量。

前置小课 前缀和

设 S[0]=0，S[i]为前 i 堆的石子总数，那么 sum(l,r)=S[r]-S[l-1]。先用 4、1、3、2 得到 S 为 0、4、5、8、10，再核对第 2 至 4 堆的和为 10-4=6。预处理 O(n)，每次区间求和 O(1)，避免把转移再多套一层求和循环。

同一依赖形状 不同代价

矩阵链乘也可按最后分界分成左右两段，但最后代价取决于矩阵维度，不是区间元素的和。若第 i 个矩阵的维度为 p[i-1]乘 p[i]，最后在 k 分开，额外乘法次数是 p[l-1]*p[k]*p[r]。把这三个维度画成两块矩阵相乘，就能解释来源。

如果石子在圆环上，任一最终合并方案都可以沿某个原始间隙展开成长度 n 的线段。把数组复制一遍，在 2n 长度上只算长度不超过 n 的区间，最后在 n 个长度为 n 的窗口里取最小值。不能直接只输出原来从 1 开始的窗口。

独立任务

用三堆 1、10、1，分别画两种第一步合并，并核对区间公式。然后只写圆环版本的答案范围，不急着写程序。解释为什么把数组复制后不能把 2n 堆全合成一堆作为答案。

第 13 章 依赖图和有向无环图

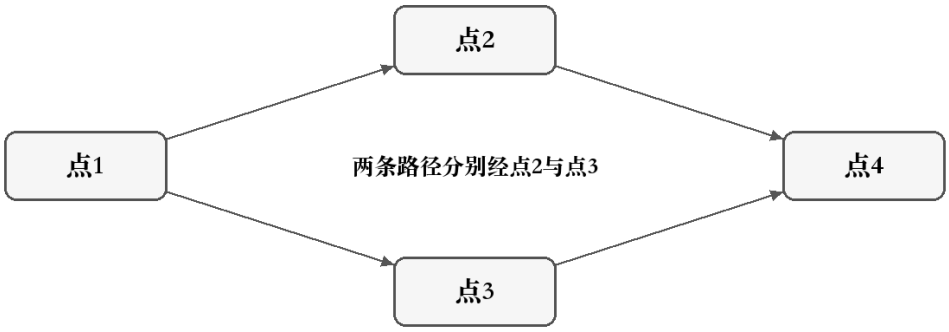
前置小课 箭头表达什么

图由点和边组成。有向边 u 到 v 表示只能沿这个方向走。有向无环图简称 DAG，意思是顺着箭头不可能绕回出发点。可以给点安排一种顺序，使每条边都从前面指向后面，这叫拓扑顺序。

找顺序的一种方法是先取没有前驱的点，删除它发出的边，再取新出现的没有前驱的点。记录每个点还有几条入边，每删除一条就减少一次。若最后没能取完所有点，图里存在环。

路径计数换了计算顺序

给定 DAG 与起点 s，问到每个点的路径数。定义 ways[v]为从 s 到 v 的路径数。ways[s]=1 表示空路径，其他为 0。按拓扑顺序处理 u，把 ways[u]加入每条出边所指的 v。



在图 1 到 2、1 到 3、2 到 4、3 到 4 中，到 4 有两条路径。若点编号被打乱，按 1、2、3、4 这样的数字顺序未必合法；编号是名字，不是依赖顺序。

如果改为从 s 出发的最长路，定义 best[v]为到 v 的最大权值和。初始化 best[s]=0，其余负 INF；只从可达 u 更新 best[v]=max(best[v],best[u]+w(u,v))。即使边权为负，DAG 的顺序仍然可用。其他源点不能凭空拥有 0 分，除非题目允许从任意点开始。

为什么复杂度常常是点数加边数

拓扑排序处理每个点一次、每条边一次。DP 也是沿每条边传递一次信息，所以总时间 O(n+m)，而不是看到“图”就认定要 $O(n^2)$。邻接表适合存稀疏图；若用邻接矩阵检查所有可能边，实际复杂度会不同。

图中的平行边是否代表不同路径必须读题。若按边序列区分，每条平行边贡献一次；若只按点序列区分，直接逐边计数可能重复。路径定义与计数对象仍然不能省略。

记忆化并不能消灭环

若某条路径可以反复绕环后再到终点，走法数甚至可能无限。带环图的最短路也需要适用的算法，不能简单照搬 DAG 最长路公式。在新题中先检查“依赖能否按有限顺序计算”，再决定是否使用这种 DP。

独立任务

把示意图中的点重新命名为 4 到 1、4 到 3、1 到 2、3 到 2，起点改为 4。先找拓扑顺序，再手推到 2 的路径数。解释按编号从小到大计算会在何处失败。

第 14 章 把有限的历史变成状态机

上一天做了什么很重要

每天有两种训练 A 与 B，各自有当天的得分。每天选一种，不能连续两天选 A，求总得分最大。最后一天选 A 或选 B，会影响下一天哪些动作合法，所以状态必须保留昨天的类型。

定义 f[i][A]与 f[i][B]分别为前 i 天结束时、当天选对应训练的最大总分。第一天直接由当天得分初始化。之后，A 只能接在 B 后面，B 可以接在 A 或 B 后面。即使训练分数为负，每天也必须选择一种，不能擅自跳过。

$$f[i][A] = f[i-1][B] + score[i][A]$$

$$f[i][B] = \max(f[i-1][A], f[i-1][B]) + score[i][B]$$

天数	A 的分数	B 的分数	以 A 结束的最好分数	以 B 结束的最好分数
1	5	2	5	2
2	6	1	8	6

天数	A 的分数	B 的分数	以 A 结束的最好分数	以 B 结束的最好分数
3	4	3	10	11

你可以把 A 与 B 画成两个小圆，允许的相邻训练之间画箭头。这种有限状态及其转移图叫状态机。DP 把“第几天”和“状态机在哪个点”放在一起。

冷却时间怎样进入状态

若选 A 后必须隔两天才能再次选 A，单纯记昨天是不是 A 就不够。可以记还需要等待的天数 r，r 只取 0、1、2：r=0 时允许选 A 并把下一天的等待数设 2；选 B 则把等待数减 1 但不低于 0。先画清楚“每天开始前”还是“每天结束后”的含义，再写数组。

若要求恰好选 K 次 A，还要记已选次数 k。若只问总次数不超过 K，答案可能要在多个 k 中比较。固定要求和可自由选择的资源数量，常常决定答案是单格还是一组格的最值。

同一动作不能在一天里连续发生

若压缩掉天数这一维，更新当天新状态时应统一读取昨天的 old，写入 next。原地写到一半又读当天结果，可能相当于一天做了两次训练。除非你已证明更新顺序安全，否则用两层数组更清楚。

独立任务

用纸片模拟“做 A 后要隔两天，另外最多做两次 A”的五天训练。设计状态，写出它是在一天开始前还是结束后。给出一对说明“只记最后一天的类型还不够”的历史。

第 15 章 多维状态和两个人同步走

多记信息之前 先找它们之间的关系

两个人同时从 n 乘 n 网格左上角出发，每步向右或向下，最后都到右下角。每个格子有非负奖励，两人经过同一格时奖励只拿一次。问总奖励最大。这里格子没有障碍；有障碍的版本需另外跳过非法状态。

若分别求两条最大路径，再把重复奖励扣掉，未必得到共同最优。第一条单独最优路径可能占掉第二条能拿到的大部分奖励。我们需要同时考虑两条路径。

先停下来想：两人都已走了 k 步时，各自坐标的行加列等于多少？如果知道 k 和一个人的行号，列号还能自己决定吗？

坐标从 1 开始，在走 k 步后有 r+c=k+2。因此状态可以记 k、r1、r2，并用 c1=k+2-r1、c2=k+2-r2 还原列。原来似乎需要四个位置下标，实际上同步步数让我们少记一个自由量。

四种来源从动作组合得到

定义 f[k][r1][r2]为两人恰好走 k 步、分别位于(r1,c1)和(r2,c2)时，已拿到的最大总奖励。上一时刻，每个人要么从上边来，要么从左边来，组合后共四种来源。

若当前行号是 r，从上边来时上一行是 r-1；从左边来时上一行仍是 r。枚举两个行号分别是否减 1，查看上一层中对应的合法状态，取最大值后加本轮奖励。若 r1=r2，同一步下列号也相同，本格奖励只加一次；否则加两格。

起点 f[0][1][1]=a[1][1]，其他状态负 INF。每一步都检查还原出的列在 1 到 n 之内。终点是 k=2n-2 且 r1=r2=n。状态总数 $O(n^3)$，每个最多四个来源；滚动 k 这一维后空间 $O(n^2)$。

为什么只检查同一步重合就够

两个人若经过同一格(r,c)，都必须在第 r+c-2 步到达它，因为只向右或向下。所以同一格的重复一定发生在同一步，不会出现一个早两步来、另一个晚两步来的情况。这个不变量让“本轮去重”正确。

如果允许等待或向左走，这个性质会失效，不能直接使用同一个模型。减少维度往往来自这样的等式或不变量，必须证明，而不是为了让数组小一点随意删维。

独立任务

在 2 乘 2 网格中填入 1、2、3、4，画出两人可能的走法，手工求共同最大收集值。再解释为什么起点和终点也只能各计算一次。写出 k=1 时所有合法的(r1,r2)组合。

第 16 章 树形动态规划先认识父子关系

前置小课 树和子树

树是连通且没有环的无向图。选一个点作根以后，每个非根点都有一个父亲，剩下向下连的点是孩子。一个点连同它下方的所有后代组成它的子树。两个不同孩子的子树彼此不重叠。

DFS 可以从根进入孩子，等孩子的子树处理完再回到父亲。在无向图中，邻接表还会包含回到父亲的边，遍历时要跳过它；没有这样做就会来回递归。树特别深时，递归栈可能不够，可用显式栈生成访问顺序，再逆序处理。

相邻的人不能同时参加

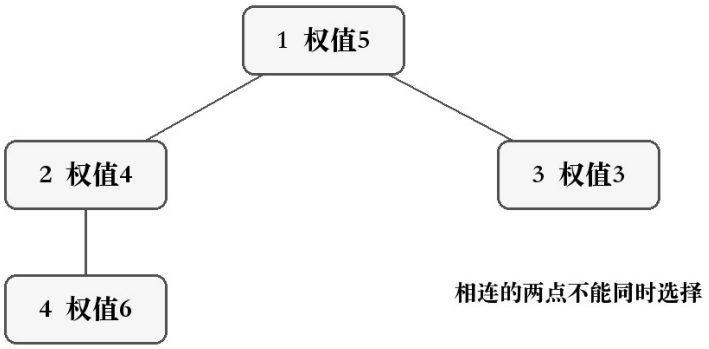
每个人有参加聚会的快乐值，关系形成一棵树。直接相连的人不能同时选，允许谁都不选，求总快乐值最大。先停下来想：某个孩子能不能参加，取决于父亲的哪一件事?

定义 dp[u][0]为 u 不参加时，u 整棵子树的最大快乐值；dp[u][1]为 u 参加时的最大快乐值。这里 0 与 1 限定的是 u 本人，不是整棵子树是否选人。

dp[u][1] = weight[u] + 所有孩子 v 的 dp[v][0] 之和

dp[u][0] = 所有孩子 v 的 max(dp[v][0], dp[v][1]) 之和

若 u 参加，每个孩子必须不参加；若 u 不参加，每个孩子可以自主选择。不同孩子子树之间没有额外边，因此可以分别求最优再相加。这是可分解的关键。如果有跨子树的限制，就不能直接独立相加。



根 1 权值 5，孩子 2 权值 4、孩子 3 权值 3，节点 2 另有孩子 4 权值 6。叶 4 的两个值是 0 和 6；节点 2 的两个值是 6 和 4；叶 3 是 0 和 3；根 1 不选时为 6+3=9，选时为 5+6+0=11。答案是根的两个状态取最大值。

叶子的 dp[u][0]=0，dp[u][1]=weight[u]。即使权值负数也可照此定义，因为允许空集，最后会选择更好的分支。每条父子边只做常数次合并，时间 O(n)。

子树里还要限制选几个

若要求恰好选 K 个人，需增加数量维：dp[u][s][k]，s 表示 u 是否选，k 表示 u 子树内恰好选几人。初始化时，不选 u 只有 k=0 可达，选 u 只有 k=1 可达。其余状态负 INF。

逐个合并孩子。当前已处理部分选了 a 人，孩子子树选了 b 人，合起来有 a+b 人；根据 u 的选择限制孩子的选择，再比较最大值。这里合并是一个小背包，每个孩子相当于提供不同人数与收益的选项。

用新的临时数组接收合并结果，避免同一个孩子贡献多次。循环上界限制在已处理子树大小、孩子子树大小与 K 之内。直接写法可用 $O(nK^2)$作粗略上界；某些模型可得到更紧界，须根据合并范围另外证明。不要把所有树上背包一概写成 O(nK)。

独立任务

在示意树上把节点 2 的权值改成 20，重新计算两种状态。解释为什么“先选全树快乐值最大的人，再删掉他的邻居”没有一般正确性保证。之后为“恰好选 2 人”列出根的可达人数与状态，不要求立即编码。

第 17 章 换根把一棵树的答案传给邻居

每个点都要作为起点

给定一棵 n 个点的树，每条边长度 1。对每个点 u，求它到全部其他点的距离总和 ans[u]。若从每个点都重新遍历整树，需要 $O(n^2)$。我们希望把已有答案改一小步，传给相邻的点。

先以 1 为根，计算 size[u]为 u 子树的点数，down[u]为 u 到本子树所有点的距离和。每个孩子 v 的子树中，所有点到 u 都比到 v 多 1，所以：

size[u] = 1 + 所有孩子 v 的 size[v] 之和

down[u] = 所有孩子 v 的 (down[v] + size[v]) 之和

按从叶到根的顺序求完后，ans[1]=down[1]。这里 size 始终按最初的根 1 定义，下一步传递时不随新的观察起点变化。

从父亲挪到孩子 哪些距离变了

从 u 走到孩子 v 作为新起点，v 原来子树里的 size[v]个点都近了 1；剩下 n-size[v]个点都远了 1。所以：

ans[v] = ans[u] - size[v] + (n - size[v])

第一遍向上收集子树，第二遍从根向下传播全树答案。两遍各处理每条边常数次，时间 O(n)。路径 1-2-3-4 的四个距离和为 6、4、4、6，可以直接列出距离核验。

加权边长 w 时，把“近 1、远 1”改成“近 w、远 w”，差值变成(n-2*size[v])*w。节点也有权重时，size 则要换成相关节点权重和；这不是原公式随意改符号，而是重新统计两边受影响的总量。

换根的一般难点

更复杂的问题需要知道“父亲那一侧给我什么贡献”，再与孩子们的贡献合并。想把信息传给孩子 v 时，常要排除 v 自己的那份贡献。加法可以减掉；最大值不能直接相减，通常要保留最大和次大，或对子节点做前后缀合并。

“先算子树，再算子树外”是一种思路，不代表每题都有同一条换根公式。必须明确传递的信息是什么，并证明去掉某个孩子后剩余信息还够用。

独立任务

在星形树中，中心 1 连接 2、3、4、5。先直接数出中心与某个叶子的距离和，再用换根式核对。向妈妈解释从中心走向一个叶子时，有多少个点变近、多少个点变远。

第 18 章 把小集合写进一个整数

前置小课 二进制是一排开关

有 n 个对象，每个对象是否已选只有两种状态。可用 n 个二进制位表示集合：第 i 位为 1 表示对象 i 在集合中。若对象编号从 0 开始，1 左移 i 位就是只打开第 i 个开关，C++写作 1<<i。

例如四个对象 0、1、2、3 中，集合{0,2}写作 0101，数值为 5。检查对象 i 是否在集合，用 mask & (1<<i)是否为 0；加入它用按位或。位运算表达的是集合关系，先用开关图解释，再写符号。位数超出 int 安全范围时需使用更宽类型并检查移位范围。

只记访问了哪些点还不够

有 n 个城市，给出两两之间非负旅行费用。固定从城市 0 出发，每个城市恰好访问一次，最后停在哪里都可以，求最小总费用。定义 dp[mask][v]为已经访问的城市集合为 mask、最后停在 v 时的最小费用。

为何要记最后位置？访问同一批城市后停在不同地方，到下一个城市的代价不同。集合相同并不意味着未来选择的代价相同，必须补上末尾 v。

初始 dp[1][0]=0，其他为 INF。从 mask 中最后的 v 出发，枚举一个不在 mask 里的城市 u，更新新集合 mask|(1<<u)的末尾 u，代价加 dist[v][u]。按集合大小增加的顺序计算最直观；按 mask 从小到大在此也可行，因为每次只加入一个原来为 0 的位，新数值必然更大。

$$\text{dp[mask 加入 u][u]} = \min(\text{原值}, \text{dp[mask][v]} + \text{dist[v][u]})$$

全部访问后，在所有末尾 v 中取最小值；若要求回起点，就在每个末尾加 dist[v][0]再比较。时间 O(2^n*n²)，空间 O(2^n*n)。n=18 时表约有 472 万格，使用 8 字节约 36 MiB，尚未计入其他内存；n 增加几位就会迅速膨胀。

行状态压缩是另一种小集合

在若干行、每行 m 列的棋盘放棋子，要求左右相邻与上下相邻的格子不能同时放。若 m 很小，可以用一个 mask 表示一行放了哪些位置。行内合法条件是 mask 与左移一位的 mask 不重叠；相邻行 a、b 合法条件是 a & b 为 0。

若有障碍，mask 还不能与障碍位重叠。定义 f[row][mask]为处理到这一行、当前行恰好如此摆放的方案数。枚举合法上一行状态，加总兼容来源。虚拟第 0 行只允许空 mask=0，方案数 1；最后对末行所有合法 mask 求和。

若行宽 m=3，没有相邻 1 的掩码为 000、001、010、100、101。先列这五个，再试上下兼容，比对 0 至 7 的数字硬背条件更清楚。若斜着也不能相邻，还要检查移位后的行间交集。

子集枚举的代价

有些题要枚举一个集合的所有子集。一个含 k 个元素的集合有 2^k 个子集；如果对全部 mask 再枚举其子集，总量是 3^n 量级，因为每个元素有“不在 mask、在 mask 不在子集、在子集”三种归属。不要把“嵌套两个 mask 循环”随意估成 4^n，也不要漏掉空子集是否需要处理。

独立任务

对三座城市，费用 0 到 1 为 2、0 到 2 为 5、1 到 2 为 1，反方向相同。列出从 0 出发访问全部城市但不返回的两条路线，核对集合与末尾状态。再说明把末尾 v 删掉会丢失什么信息。

第 19 章 数位动态规划把大范围拆成短数字

从一到大数 为什么还能做表

统计 0 到 R 中，十进制表示不含数字 4 的整数个数。若 R 很大，逐个检查太慢，但 R 最多只有十几或二十位。我们可以从最高位开始决定每一位，使构造出的数字不超过 R。

先停下来想：R=352。如果前面已经填成 2，后面的两位还受 5 和 2 限制吗？如果前面填成 35，最后一位的上限又是多少？

若前缀已严格小于上界前缀，后面的每一位都可从 0 到 9；若还相等，下一位只能不超过上界该位。这需要一个 tight 标记，表示前缀是否仍贴着上界。

把状态定义完整

用 solve(pos,tight,started,extra)表示从第 pos 位开始，已知前缀条件下合法的后缀个数。started 表示是否已经出现第一个非零有效数字。extra 记录题目真正需要的额外历史，例如上一位是什么，或者数字和除以 K 的余数。

“不含 4”本身可在等长补前导零的表示里判断，且 0 视为合法时可以简化 started。但若题目涉及数字长度、相邻数字、0 出现次数，前导零与真实的数字 0 必须分清，所以先学完整版本。

每次枚举 d 从 0 到上限。新 tight 只有在旧 tight 为真且 d 等于上界本位时才为真；新 started 由旧 started 或 d 不为 0 决定。若仍未开始，就走前导零分支，不把它误记成真实数字。若已开始，则检查数字是否合法，并更新 extra。

当 pos 越过最后一位时，检查整个数是否满足条件。若 started 仍为假，这条分支表示数 0，要按题目对 0 的定义单独判定。它不是很多个不同的全零数。

范围查询和缓存

设 F(R)统计 0 到 R 的答案，区间[L,R]的答案是 F(R)-F(L-1)，并约定 R<0 时 F(R)=0。一般缓存 tight=false 的状态，因为此时后缀不再受当前上界具体数字限制。缓存键必须包含会影响未来的全部状态。

若把 tight=true 的结果也缓存，必须保证缓存只服务于同一个上界，或把相应上界信息放入键中。调用 F(R)和 F(L-1)时复用旧缓存却忘了上界已变，容易得到看似合理的错误答案。

一次完整的小例子

R=25 且不含 4。按十位分组：补零十位的 0 至 9 中有 9 个合法数；10 至 19 有 9 个；20 至 25 排除 24，剩 5 个，总共 23 个。全程只是在数分支，没有把每个大整数逐一访问。

如果改成“相邻两个有效数字不能相同”，需要记上一位 last，并用一个特殊值表示还未开始。若改成“数字和能被 3 整除”，需要记余数 rem，新余数为(rem+d)%3。题目条件决定 extra，不能给所有数位题套同一个状态。

独立任务

设计“0 到 R 中有效数字不含 4，且数字和是 3 的倍数”的状态。先列 0 到 20 的合法数字作为核验，不看第四册。说明数字 0 是否应该计入，以及前导零是否会制造多种表示。

第 20 章 概率与期望也是对子问题的合并

前置小课 平均要按概率加权

若一次游戏一半概率得 0 分，一半概率得 10 分，平均得分是 5。若两种结果概率不同，就不能简单除以 2，而要分别乘出现概率再相加。这个长期平均叫期望，不表示每次真的会得到 5 分。

本章只讨论有限状态、转移概率已知的简单模型。遇到概率知识还陌生时，可以先反复模拟或用表格列所有小结果，再学习公式。概率转移用加权和，最值转移用 min 或 max，不能混用。

没有循环依赖的期望

从数轴位置 x<n 出发，每次等概率向前 1 或 2，达到或超过 n 就停止。求期望还要走几步。定义 E[x] 为从 x 开始到结束的期望剩余步数。x>=n 时 E[x]=0，因为已经结束。

在 x<n 时一定先走一步，然后以相等概率来到 x+1 或 x+2，所以 E[x]=1+(E[x+1]+E[x+2])/2。依赖更大的位置，因此从 n-1 向 0 倒着算。这里倒序不是例外，而是同一条“先算来源”的原则。

例如 n=3：E[3]=E[4]=0，E[2]=1，E[1]=1.5，E[0]=2.25。期望可以不是整数，即使每次真实步数都是整数。输出小数位数与允许误差要按题目要求处理。

原地停留会产生方程

若从 x 有概率 p 原地停留、概率 1-p 前进一格，每次尝试都算一步，则 E[x]=1+p*E[x]+(1-p)*E[x+1]。把含 E[x]的项移到一边，在 p<1 时得到 E[x]=1/(1-p)+E[x+1]。

p=1 且尚未结束时，永远不会前进，不能做除以 0 的计算，也没有有限的期望结束时间。更复杂的环可能需要解联立方程，不能因为写出了递归函数就认为已经得到 DP。

概率状态与计数状态的差别

统计有多少路径，通常每条合法路径贡献 1；统计到达概率，每条转移贡献来源概率乘转移概率。不同长度的路径未必等概率，所以“合法路径数除以总路径数”并不总是正确。

当题目用模意义表示概率时，涉及模逆元及其存在条件，是额外的数论知识。本章的实数期望不能简单把除法改成整数除法或随意取模。

独立任务

把前进 1 或 2 的概率改成四分之三与四分之一，终点仍为 3。自己推导三个非终止状态的期望，不靠记忆改公式。再解释“每次先付一步”和“到达终点就不用再走”的边界怎样进入计算。

第 21 章 优化之前先找到重复劳动

建立慢方法作为参照

优化的起点是一份正确、可解释的朴素 DP。先数状态有多少、每格找来源花多少时间、存了多少层。瓶颈可能是区间求和、枚举前驱、枚举容量、保存所有层，处理方式各不相同。

每次优化后，用相同的小输入核对优化前后整张表或最终答案。新方法快但与旧方法不一致时，先保留失败样例，不要立刻把旧方法删掉。要证明优化保留了哪些来源，为什么丢掉的来源永远不会更好。

前缀和与前缀最值

如果转移是 f[i]=sum(f[j])，j 从 max(0,i-K)到 i-1，可以用累计和得到这段总和，避免每次重新相加。若取模，做区间相减后须把负值正规化到 0 至 M-1。

若转移是 f[i]=w[i]+max(f[j])，j 取 0 到 i-1，可以维护一个前缀最大值，一次读出答案。但如果范围变成[i-K,i-1]，旧最大值可能已经过期，不能仅用一个历史最大值。第 22 章将处理这个窗口。

用价值当下标而不是重量

若 0/1 背包容量 W 很大，但所有非负整数价值总和 V 很小，可以定义 g[v]为恰好获得总价值 v 所需的最小重量。初始 g[0]=0，其余 INF，每件物品按价值倒序更新最小重量，最后找 g[v]<=W 的最大 v。

时间 O(nV)，与按重量的 O(nW)相比，关键是选择更小的可枚举范围。这叫改变状态坐标，不是把原表粗暴截短。负价值或其他附加条件下，需要重新检查下标范围与转移。

最长上升子序列怎样变快

第一册的 O(n²)算法枚举每个元素前面所有更小的值。一种优化保留 tail[len]，表示已扫描前缀中长度为 len 的严格上升子序列，其最小可能结尾。长度相同，结尾更小不会减少未来可接的更大数，所以较大结尾可以被替代。

处理新数 x 时，找到第一个 tail 位置，其值大于或等于 x，用 x 替换；若没有这样的位置，就把最长长度增加 1。tail 按长度严格递增，可二分查找。严格上升使用第一个>=x 的位置；不下降子序列才改为第一个>x 的位置。

扫描 3、1、2、2、4 时，tail 依次是[3]、[1]、[1,2]、[1,2]、[1,2,4]。它告诉我们最优长度，不保证当时 tail 中的全部数字来自同一条真实子序列；要输出方案还需记录原下标与前驱。

这个优化正确的关键是“同长度下结尾越小越有利”。若题目加了颜色限制、距离限制或收益目标，该支配关系可能不足，不能直接用 tail 替代原状态。

数据结构维护前驱最优值

若转移需查“所有值小于 a[i]的前驱状态”的最大值，可先把数值离散化成相对排名，再用树状数组或线段树查询前缀最大值。处理 a[i]前查询排名 1 至 rank-1，再把本状态更新到 rank，可保持严格小于。

树状数组是一组覆盖不同长度区间的小盒子。前缀查询把目标前缀拆成不重叠的几个盒子；单点更新修改包含该点的相关盒子。需要先学会 lowbit 及其区间含义，再实现。这里只做取更大值的单调更新时可用前缀最大树状数组；一般删除或把值改小不能直接套同一写法。

独立任务

序列 2、2、3，分别手推严格上升与不下降的 tail 过程。再把“枚举所有更小前驱”的状态写成一次数据结构查询，指出是否应该查询当前值的排名本身。不会二分或树状数组时，先保留 O(n²)程序作为验证依据。

第 22 章 单调队列维护会过期的候选

从有限步长跳跃出发

有位置 0 至 n，每次向前跳 1 至 K 格，落到位置 i 会获得 a[i]分，必须到 n。设 f[i]为到 i 的最大总分，f[0]=0。每个前驱 j 必须在 max(0,i-K)到 i-1 之间。

$$f[i] = a[i] + \max(f[j]) \quad \max(0,i-K) <= j < i$$

朴素写法 O(nK)。当 i 向右移动时，合法前驱形成滑动窗口：左边最旧位置过期，右边新状态加入。我们想快速知道窗口中最大的 f[j]。

为什么某些候选可以永久删掉

假设两个候选下标 p<q，而 f[p]<=f[q]。q 得分至少一样高，且比 p 更晚离开未来窗口，所以 p 在未来任何二者都合法的时刻都不会更好；当 q 过期时 p 早已过期。因此 p 可以从候选队列中删掉。

队列中的下标从前到后增大，f 值从前到后严格下降或不增，取决于相等时的删除策略。队首就是当前最大值。这里若只求最大值，可以在相等时保留更晚的 q；若还要统计最优方案数或输出特定路径，相等候选不能未经处理就丢弃。

每个位置的处理顺序

先把队首中下标小于 i-K 的候选删除
用当前队首求 f[i]
从队尾删除所有值不大于 f[i] 的候选
把下标 i 放入队尾

求 f[i]前不能先把 i 加入队列，否则会把当前状态当成自己的前驱。初始队列放位置 0；若存在无法到达的状态，就只把可达状态加入，并处理队列为空的情况。

手推一次队列

取 K=2，a[1..4]=2、-1、3、1。f 依次为 0、2、1、5、6。处理 i=3 前，前驱只允许 1 和 2，位置 0 已过期；值 2 比值 1 大，所以从位置 1 到 3 得到 5。新位置 3 的值 5 优于旧候选，且更晚过期，因此它们可以退出。

每个下标入队一次、最多从队首或队尾退出一次，总操作数 O(n)。虽然代码有 while 循环，总工作量并非 O(n²)，这叫均摊分析：给每个元素数一生中的入队与出队次数。

多重背包怎样用上它

多重背包当前物品重量 w、价值 v、最多 s 件。对某个余数 r，把容量写成 r+t*w。旧层到新层的转移可以改写为：new[r+t*w]等于 t*v，加上 old[r+q*w]-q*v 在 max(0,t-s)<=q<=t 中的最大值。

推导来自使用 t-q 件当前物品。固定余数后，q 形成长度至多 s+1 的滑动窗口，所以可以用单调队列。注意这里 q=t 也合法，代表当前物品一件不拿；和前面的跳跃题 j<i 不一样。旧层必须保留，按余数组分别处理。

本章首先要求你独立实现跳跃问题；多重背包优化在慢版与拆分版都稳定后再做。每次都从真实转移推出窗口，不能只因为看到“最大值”就放一个队列。

独立任务

对 K=2、a=5、-100、-100、5 画出每一步合法窗口。找出全局历史最大值已经过期却仍被错误使用的时刻。解释只维护一个最大数为什么不够。

第 23 章 进一步优化要先证明条件

把所有候选看成直线

考虑 f[i]=min(f[j]+(S[i]-S[j])²+C)，0<=j<i，f[0]=0。S 是某个非负序列的前缀和，C 是固定分段费用。直接枚举 j 是 O(n²)。先把平方展开，分离只与 i 有关和只与 j 有关的部分。

$$f[i] = S[i]*S[i] + C + \min((-2*S[j])*S[i] + f[j] + S[j]*S[j])$$

对每个 j，设直线斜率 m[j]=-2*S[j]，截距 b[j]=f[j]+S[j]²。求 f[i]变成在 x=S[i]处找这些直线中最低的一条。新状态算完后，它也变成一条新直线，供后续使用。

先停下来想：为什么不能在计算 f[i]之前，把由 f[i]生成的线放进去？这会把 j=i 加入来源，与 j<i 的规则冲突，并形成自我依赖。

单调凸包何时成立

由于原数组非负，S 不下降，加入的斜率不增加，查询 x 也不下降。对最小值问题，可以维护下凸包，并让查询指针只向一个方向移动。相同斜率只保留截距更小的一条；若需要方案计数，要额外保留并列信息。

对斜率严格递减的三条线 a、b、c，若交点 x(a,b)不小于 x(b,c)，中间线 b 没有独占最优的区间，可以删除。因为 m[a]-m[b]与 m[b]-m[c]都为正，这个比较可以通过交叉乘法完成。

$$(b[b]-b[a])*(m[b]-m[c]) >= (b[c]-b[b])*(m[a]-m[b])$$

这里 m 与 b 分别是斜率和截距数组；字母 b 同时作为中间线标签时容易读混，编码可改名为 slope、intercept 与索引 u、v、w。交叉乘积可能超出 long long，必须根据范围选择更宽的中间类型。不要为了省几个字造成变量职责混乱。

若 S 可能下降，斜率和查询点的单调性就可能消失，普通单调队列式凸包不能照搬。可考虑适用的数据结构如李超树或按条件使用二分查询，但需要单独学习和证明。这一页说明适用边界，不把名字本身当作算法已经学会。

一个可核验的例子

取 S 为 0、1、3、4，C=2。朴素枚举得到 f[1]=3，f[2]=9，f[3]=12。分别把 j=0、1、2 变成直线，在 x=1、3、4 处查询，验证两种计算完全相同。先做这一步，再谈删除冗余线。

在会写优化前，至少能自己完成平方展开、指出每条线代表哪个决策、解释平行线处理，并用小数据与 O(n²)对照。对公式不熟时，停在朴素解也是有价值的正确进展。

分治优化依赖决策单调性

另一类模型是把前 j 个元素分成 g 段，定义 D[g][j]=min(D[g-1][k]+cost(k+1,j))，k 在 g-1 至 j-1。若已证明最优分界 opt[g][j]随着 j 增加不下降，就能用分治减少候选搜索范围。

先计算当前区间中点 mid，在允许的候选范围里找最优 k。左半边的最优分界不大于这个 k，右半边的不小于这个 k，于是递归处理两边，各自缩小搜索范围。一层 DP 的常见复杂度降为 O(n log n)，总体 O(Gn log n)，但前提是代价能快速计算且决策单调性成立。

选并列最优时要统一取最小或最大分界，避免证明与实现使用不同的 opt 定义。仅在几个样例中看到分界递增，不构成证明；应使用题目代价的性质、已证明定理，或保留朴素方法。

区间优化不是把上一个方法搬过去

第 12 章非负石子合并满足一类更强条件，可用 Knuth 优化把最优分界限制在 opt[l][r-1]到 opt[l+1][r]之间，从而做到 $O(n^2)$。它的区间包含关系与四边形不等式条件不同于上一段的分层分治优化。

这一优化在非负区间和代价下有适用依据；若代价被改成任意函数、元素允许负数，不能直接假设成立。提高阶段的目标是先能写出精确条件和慢版，再决定是否继续推导定理。较难优化并非所有 DP 题的必经步骤。

把分治优化具体做一层

假设已经证明决策单调性，当前要计算本层 j 从 L 至 R 的答案，候选分界在 lo 至 hi。取 mid 为区间中点，只枚举 lo 至 min(hi,mid-1)中的合法 k；对恰好 g 段，还应与 k>=g-1 以及旧层可达性共同检查。找出最优分界 best 后，左半区候选上界收缩到 best，右半区候选下界提高到 best。

一层分治的流程

求当前层的区间 L 至 R，候选范围 lo 至 hi：
  如果 L 大于 R，结束这一分支
  取中点 mid
  在合法候选中，读上一层并找最优分界 best
  保存当前层 mid 的答案
  递归处理 L 至 mid-1，候选 lo 至 best
  递归处理 mid+1 至 R，候选 best 至 hi

这段流程的正确性来自候选范围确实包含对应位置的最优分界。若某段没有合法候选，应保持不可达，并按具体分层边界安排计算，不能拿一个未赋值的 best 继续递归。常见实现直接令本层 j 从 g 开始，上一层只允许至少 g-1 个元素。

为什么可能满足单调性？例如代价是非负序列一段总和的平方。把两段端点向外扩展时，交叉增加量由非负量的乘积决定，满足相应的 Monge 不等式；加上只与分界 k 有关的上一层值后，这个比较关系仍保留，便能推出最优分界不逆向跳动。新代价必须重新核对性质，不能只认出“分段”两个字。

把区间优化的条件写出来

对石子合并的区间代价 w(l,r)=sum(l,r)，若 a<=b<=c<=d，非负石子保证 w(b,c)<=w(a,d)，因为外区间包含更多非负石子；同时 w(a,c)+w(b,d)=w(a,d)+w(b,c)，因为每一段的计数次数相同。后一个等式自然满足需要的四边形不等式。

Knuth 优化定理在这类条件下给出最优分界的夹逼关系。取并列最优中最大的分界，令 opt[i][i]=i；求区间[l,r]时，仅枚举从 opt[l][r-1]到 min(r-1,opt[l+1][r])的 k，仍然能保留一个最优方案。按区间长度递增，两个夹逼边界已经算好。

以前面的 4、1、3、2 为例，长度 2 的分界分别为 1、2、3；长度 3 中 opt[1][3]=1、opt[2][4]=3；长度 4 仍会选 k=2，费用 20。注意不同区间的 opt 不是看起来总递增的一串数字，保证的是上面的两条区间关系。

总枚举量降为 $O(n^2)$的原因是固定长度时，相邻区间的候选边界会相互抵消，总跨度只有 O(n)；共有 O(n)种长度。并不是每个区间候选都固定只剩两个。实现前可以把慢版的全部 opt 打印出来，确认索引约定与不等式对应。

本节给出了条件、使用步骤和例子。若想将定理迁移到新代价，需要进一步证明该代价满足这些条件；本书不要求初学阶段证明所有优化定理本身。能把“已证明的定理如何用于本题”和“尚未证明的猜测”分开，是提高组需要的严谨习惯。

独立任务

对 S=0、2、3、6，C=1，先计算朴素 DP，再列每个决策对应的直线。另构造一个包含负增量的 S，指出哪一项单调性首先失效。能识别“这个优化不适用”，也是掌握的一部分。

第 24 章 面对没有标签的新题

从一句新条件找到状态缺口

设有一排展品，每件有价值；不能选相邻两件；最多选 K 件；还要求所有选中价值之和除以 3 余 1。不要一次把全部符号写到纸上。先解决“不能相邻”，再问每个新增条件需要记住什么。

位置 i 决定已经看过哪些展品；上一件是否选影响下一件能否选；已选数量决定是否超额；价值和的余数决定终点是否满足要求。可以定义 f[i][last][k][rem]保存最大价值，或者按题目所求改成方案数。目标类型决定保存的量，限制决定状态维度。

如果要求最大价值，余数只能作为约束信息，不代表可以把真实价值也取模后比较。最后只能在合法数量、合法末尾状态与余数 1 的格子里取最大值。没有合法方案应报告无解，不可用 0 冒充。

建模和简化的完整工作顺序

先写一个能正确表达条件的状态，估算状态数。若太大，检查是否有维度能由其他量算出，是否存在可替换的较差历史，是否能换一个更小的量作下标。仍然太大，再寻找转移优化或另一种算法。

同一问题可能有多种正确状态。例如不相邻选择，可以用“前 i 件的最优”跳过 i-1，也可以显式记上一件是否选。更容易证明、更方便添加新限制、在数据范围内可运行的定义，就是合适的定义。

不要仅凭数据范围猜题型。n 很小提示可以枚举或集合状态，容量很小提示背包，但这些只是线索。真正的模型必须解释方案与状态之间怎样对应，转移为什么完整且合法。

同时求最优值与最优方案数

有些题既问最好能得到多少分，又问有多少种方法达到这个最好分。可以在每个状态保存一对值 best 与 count。发现更好的候选时，用新 best 替换，并把 count 改成该来源的方案数；发现并列最优时，把方案数加进来；更差候选不贡献。

这仍要求来源分类不重复。如果同一完整方案能从两种转移走到同一状态，直接把 count 相加会重计。涉及多个相互独立子部分时，组合方案数可能需要相乘，然后再对不同决策相加。最优值用真实整数比较，计数只在题目要求时取模。

如果还要求输出一条最优方案，在二维或多维表中记录被选中的来源，沿着严格变小的规模逆向追溯。计数、最优值和方案记录是三种职责，可以一起保存，但不能让同一个变量在不同循环里偷偷换含义。

比赛中的可执行策略

先读完整题目与数据范围，列出小规模正确做法。若完整解暂时没有出来，检查是否能为特定限制实现已证明正确的部分解；部分分条件以题目为准。保留能运行的版本，再逐步优化，避免把一个正确程序改成尚未验证的复杂程序。

提交前检查数组大小、类型、边界、不可达、取模、复杂度和输入输出要求。调试输出要移除。遇到未知失败，缩小样例并找第一格偏差；遇到超时，统计实际循环范围，不能只给代码贴一个想象的 O(n)标签。

学会迁移需要怎样的练习

先做改变一项规则的近迁移，如背包从最多改恰好、网格计数改最值。随后做题面不同但状态相同的远迁移，如训练选择与机器冷却。最后混合不同算法的题，不告诉你是不是 DP，让你决定是否需要 DP。

每次总结由你自己写：原来漏记了什么、哪一个反例让我发现、下次先检查哪句话。教练可以指出遗漏，不能替你编写“我已经懂了”。不会做一题时，记录的是尚缺的一项能力，而不是给自己贴“不会 DP”的标签。

第二册验收与学习边界

先完成第三册 B 组，再按前置条件选择 C 组。树形、集合、数位与优化应分别留下独立作品；不把做过一题就当作整个专题掌握。通过同模型变式后，再用不标模型的综合题检验。

本书覆盖从基础到提高组的主要 DP 建模方法，并介绍进一步优化的适用边界。它不穷尽所有竞赛构造，也不替代树状数组、线段树、图论、字符串、数论等独立专题。出现新结构时，仍要回到状态、依赖与证明，而不是寻找一份无限长的模板清单。

本册资料说明

本册讲解与训练为原创教学编排。范围参照中国计算机学会发布的 NOI 大纲 2025 年修订版，核对日期 2026 年 10 月 12 日；进阶章节的顺序与难度分层是教学建议。不同考试年度的实际题目和规则以官方发布为准。

https://www.noi.cn/xw/2025-04-18/841584.shtml

https://www.noi.cn/upload/resources/file/2025/04/18/NOI_Syllabus_Edition_2025.pdf